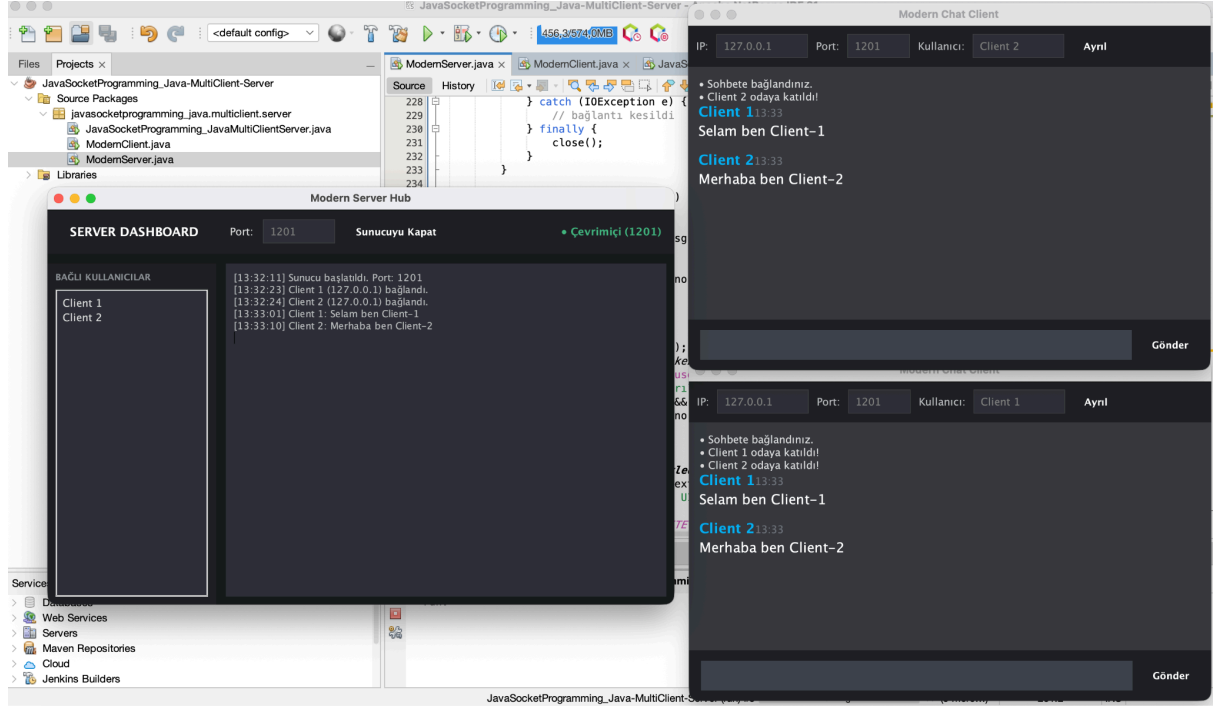


ÇOKLU İSTEMCİ DESTEKLİ SOKET TABANLI SOHBET SİSTEMİ



PROJE RAPORU

Proje Adı: Modern Multi-Client Chat Application

Geliştirici: Muhammed Emin Korkunç

E-posta: muhammedeminkorkunc@gmail.com

LinkedIn: <https://www.linkedin.com/in/muhammed-emin-korkunc-100ba2215>

GitHub: <https://github.com/muhammedkorkunc>

Paket: javasocketprogramming_java.multiclient.server

Programlama Dili: Java

Java Sürümü: JDK 21

Grafiksel Arayüz: Java Swing

Ağ Teknolojisi: TCP/IP Socket Programming

ÇOKLU İSTEMCİ DESTEKLİ SOKET TABANLI SOHBET SİSTEMİ	1
PROJE RAPORU	1
1. PROJE ÖZETİ VE AMACI	4
2. PROJENİN GENEL MİMARİSİ	5
3. KULLANILAN TEKNOLOJİLER	6
4. MODERNCLIENT SINIFI	7
4.1. İstemci Soketi	7
4.2. DataInputStream ve DataOutputStream	7
4.3. Kullanıcı Adının Gönderilmesi	8
4.4. Arka Plan Listener Thread	9
4.5. Mesajların Ayırıştırılması	9
4.6. HTML Destekli Sohbet Arayüzü	10
4.7. Mesaj Gönderme	11
4.8. Bağlantı ve Ayrılma İşlemleri	11
5. MODERNSERVER SINIFI	12
5.1. ServerSocket Kullanımı	12
5.2. Çoklu İstemci Desteği	13
5.3. Client Listesi	14
5.4. ClientHandler Sınıfı	14
5.5. Kullanıcı Adının Alınması	15
5.6. Broadcast Mekanizması	15
5.7. Sistem Mesajları	16
5.8. Kullanıcı Mesajları	16
6. SUNUCU GRAFİKSEL ARAYÜZÜ	17
6.1. Üst Kontrol Paneli	17
6.2. Bağlı Kullanıcılar	18
6.3. Sistem Logları	18
7. RENK VE ARAYÜZ TASARIMI	19
8. KARŞILAŞILAN PROBLEMLER VE ÇÖZÜMLER	19
8.1. GUI'nin Donması	20
8.2. Tek İstemci Problemi	20
8.3. İstemciler Arası Mesaj Dağıtımı	20
8.4. Swing Arayüzünün Güvenli Güncellenmesi	21
9. VERİ İLETİŞİM PROTOKOLÜ	21
Sistem Mesajı	21
Kullanıcı Mesajı	21
10. PROGRAMIN ÇALIŞTIRILMASI	22
10.1. Sunucunun Çalıştırılması	22
10.2. İstemcinin Çalıştırılması	22
10.3. Çoklu İstemci Testi	23
11. ÖRNEK ÇALIŞMA SENARYOSU	24
12. PROJENİN AVANTAJLARI	25
13. PROJENİN SINIRLILIKLARI	25
14. GELECEK GELİŞTİRMELER	26
14.1. Dosya ve Resim Transferi	26
14.2. Özel Mesajlaşma	26
14.3. SQLite Veritabanı	26
14.4. TLS/SSL Desteği	27
15. SONUÇ	27
16. SINIFLARIN GENEL GÖREVLERİ	28
17. PROJE BİLGİLERİ	29
18. GELİŞTİRİCİ	29
EK: PROJE DOSYALARI	29

1. PROJE ÖZETİ VE AMACI

Bu projenin temel amacı, TCP/IP soket programlama teknolojisi kullanılarak birden fazla istemcinin aynı anda bağlanabildiği, istemci-sunucu mimarisine sahip masaüstü tabanlı bir sohbet uygulaması geliştirmektir.

Proje Java programlama dili kullanılarak geliştirilmiş ve grafiksel kullanıcı arayüzü için Java Swing teknolojisinden yararlanılmıştır.

Sistem iki temel uygulamadan oluşmaktadır:

- ModernServer
- ModernClient

ModernServer, istemcilerin bağlantılarını kabul eden ve istemciler arasındaki mesaj trafiğini yöneten merkezi sunucu uygulamasıdır.

ModernClient ise kullanıcıların sunucuya bağlanmasını, mesaj göndermesini ve diğer kullanıcılar tarafından gönderilen mesajları görüntülemesini sağlayan istemci uygulamasıdır.

Projenin temel amacı, aynı anda birden fazla kullanıcının tek bir merkezi sunucu üzerinden gerçek zamanlı olarak iletişim kurabilmesini sağlamaktır.

Bu amaç doğrultusunda sunucu tarafında her istemci bağlantısı için ayrı bir ClientHandler nesnesi oluşturulmuş ve her bağlantı bağımsız bir Thread içerisinde çalıştırılmıştır.

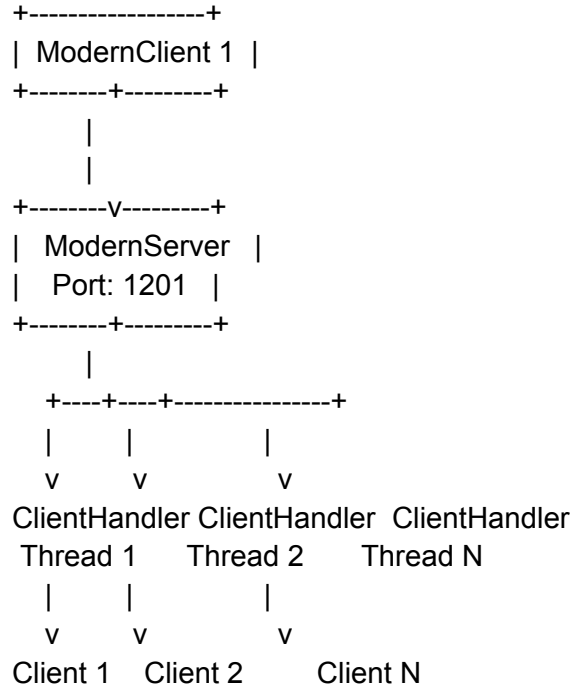
Ayrıca kullanıcı deneyimini geliştirmek amacıyla modern koyu temalı bir grafiksel arayüz tasarlanmıştır.

İstemci tarafında sohbet mesajlarının daha düzenli ve görsel olarak daha anlaşılır şekilde gösterilebilmesi için JEditorPane ve HTML biçimlendirme kullanılmıştır.

2. PROJENİN GENEL MİMARİSİ

Proje, merkezi istemci-sunucu (Client-Server) mimarisine göre tasarlanmıştır.

Sistemin genel çalışma yapısı aşağıdaki gibidir:



Sunucu merkezi bir iletişim noktası olarak görev yapmaktadır.

İstemciler doğrudan birbirleriyle iletişim kurmak yerine öncelikle sunucuya bağlanmaktadır.

Bir istemci tarafından gönderilen mesaj sunucuya ulaşmakta ve sunucu tarafından broadcast () metodu kullanılarak bağlı istemcilere dağıtılmaktadır.

Bu yapı sayesinde tüm istemcilerin merkezi bir sistem üzerinden haberleşmesi sağlanmaktadır.

3. KULLANILAN TEKNOLOJİLER

Projede aşağıdaki teknolojiler ve Java sınıfları kullanılmıştır:

Teknoloji / Sınıf	Kullanım Amacı
Java JDK 21	Programlama dili ve çalışma ortamı
Java Swing	Grafiksel kullanıcı arayüzü
Socket	İstemci-sunucu bağlantısı
ServerSocket	Sunucu tarafında bağlantı kabulü
TCP/IP	Güvenilir ağ iletişimi
Thread	Eşzamanlı istemci yönetimi
DataInputStream	Ağ üzerinden veri okuma
DataOutputStream	Ağ üzerinden veri gönderme
JEditorPane	HTML destekli sohbet ekranı
JTextField	Kullanıcı girişi
JTextArea	Sunucu log ekranı
JList	Bağlı kullanıcıların gösterimi
Collections.synchronizedList	Eşzamanlı istemci listesi
SwingUtilities.invokeLater	Swing arayüzünün güvenli güncellenmesi

4. MODERNCLIENT SINIFI

ModernClient sınıfı, sistemin istemci tarafını oluşturmaktadır.

Bu sınıf JFrame sınıfından türetilmiştir ve kullanıcıya grafiksel bir sohbet arayüzü sunmaktadır.

Sınıf içerisinde ağ bağlantısı, mesaj gönderme, mesaj alma, bağlantı kurma ve bağlantıyı sonlandırma işlemleri gerçekleştirilmektedir.

4.1. İstemci Soketi

İstemcinin sunucuya bağlanabilmesi için Java'nın Socket sınıfı kullanılmaktadır.

Bağlantı sırasında kullanıcı tarafından girilen IP adresi ve port bilgileri kullanılmaktadır.

Kod içerisindeki varsayılan değerler:

IP Adresi : 127.0.0.1
Port : 1201

Bağlantı işlemi aşağıdaki yapı ile gerçekleştirilmektedir:

```
socket = new Socket(host, port);
```

Bu işlem sonucunda istemci ile sunucu arasında TCP bağlantısı oluşturulmaktadır.

4.2. DataInputStream ve DataOutputStream

İstemci ile sunucu arasındaki veri aktarımı için:

DataInputStream
DataOutputStream

sınıfları kullanılmaktadır.

Sunucudan veri almak için:

```
dis = new DataInputStream(socket.getInputStream());
```

Sunucuya veri göndermek için:

```
dout = new DataOutputStream(socket.getOutputStream());
```

kodları kullanılmaktadır.

Mesaj gönderilirken:

```
dout.writeUTF(text);  
dout.flush();
```

işlemleri gerçekleştirilmektedir.

4.3. Kullanıcı Adının Gönderilmesi

İstemci sunucuya bağlandıktan sonra kullanıcı adı sunucuya gönderilmektedir.

Kod:

```
dout.writeUTF(user);  
dout.flush();
```

Sunucu tarafındaki `ClientHandler` ise bu kullanıcı adını:

```
username = dis.readUTF();
```

koduyla almaktadır.

Bu sayede sunucu hangi istemcinin hangi kullanıcı adına sahip olduğunu takip edebilmektedir.

4.4. Arka Plan Listener Thread

Sunucudan gelen mesajların dinlenmesi için istemci tarafında ayrı bir Thread kullanılmaktadır.

Kod içerisinde:

```
new Thread(() -> {  
    while (connected) {  
        try {  
            String raw = dis.readUTF();  
            ...  
        } catch (IOException e) {  
            break;  
        }  
    }  
    disconnect();  
}).start();
```

yapısı kullanılmaktadır.

Buradaki temel amaç, readUTF () işleminin kullanıcı arayüzünün çalıştığı ana Thread'i engellemesini önlemektir.

Çünkü readUTF () sunucudan veri gelmesini bekleyebilen bir işlemdir.

Bu işlemin ayrı bir Thread içerisinde çalıştırılması sayesinde kullanıcı arayüzü mesaj beklerken donmamaktadır.

4.5. Mesajların Ayırıştırılması

Sunucudan gelen veriler iki temel biçimde ele alınmaktadır.

Sistem mesajları:

SYS:[mesaj]

Kullanıcı mesajları:

MSG:[kullanıcı]:[mesaj]

İstemci tarafında:

```
if (raw.startsWith("SYS:")) {
```

```
addSystemMessage(raw.substring(4));  
}
```

kodu ile sistem mesajları ayrıştırılmaktadır.

Kullanıcı mesajlarında ise:

```
String[] parts = raw.split(":", 3);  
addChatMessage(parts[1], parts[2]);
```

kullanılmaktadır.

Buradaki `split(":", 3)` kullanımı sayesinde mesajın en fazla üç parçaya ayrılması sağlanmaktadır.

4.6. HTML Destekli Sohbet Arayüzü

Sohbet ekranında standart bir `JTextArea` yerine `JEditorPane` kullanılmıştır.

Kod:

```
chatPane = new JEditorPane();  
chatPane.setContentType("text/html");  
chatPane.setEditable(false);
```

şeklindedir.

Bu yapı sayesinde sohbet alanında HTML ve CSS benzeri biçimlendirmeler kullanılabilir.

Mesajlarda aşağıdaki görsel özellikler uygulanmıştır:

- Kullanıcı adı
- Saat bilgisi
- Mesaj içeriği
- Sistem mesajları
- Farklı renkler
- Koyu tema
- Mesajlar arasında boşluk

Mesaj saati:

```
String time = new SimpleDateFormat("HH:mm").format(new Date());
```

koduyla alınmaktadır.

4.7. Mesaj Gönderme

Kullanıcı mesaj alanına bir mesaj yazdıktan sonra Gönder butonuna basarak mesajını sunucuya gönderebilmektedir.

Ayrıca Enter tuşuna basıldığında da mesaj gönderilmektedir.

Bu işlem:

```
if (e.getKeyCode() == KeyEvent.VK_ENTER) {  
    sendMessage();  
}
```

koduyla sağlanmaktadır.

Mesaj gönderme işlemi:

```
private void sendMessage() {  
    String text = msgInput.getText().trim();  
    if (text.isEmpty() || !connected) return;  
  
    try {  
        dout.writeUTF(text);  
        dout.flush();  
        msgInput.setText("");  
    } catch (IOException ex) {  
        addSystemMessage("Mesaj iletilemedi.");  
    }  
}
```

şeklinde gerçekleştirilmektedir.

4.8. Bağlantı ve Ayrılma İşlemleri

İstemci bağlantı kurduğunda:

Bağlan

butonunun metni:

Ayrıl

olarak değiştirilmektedir.

Bağlantı sonlandırıldığında ise tekrar:

Bağlan

haline gelmektedir.

Bağlantı kapatılırken:

`socket.close();`

kullanılmaktadır.

5. MODERNSERVER SINIFI

ModernServer sınıfı sistemin merkezi sunucu uygulamasıdır.

Bu sınıf da JFrame sınıfından türetilmiştir ve sunucu yönetimi için grafiksel bir kontrol paneli sunmaktadır.

Sunucunun temel görevleri:

- Belirlenen portu dinlemek
- İstemci bağlantılarını kabul etmek
- İstemcileri yönetmek
- Kullanıcı isimlerini takip etmek
- Gelen mesajları almak
- Mesajları diğer istemcilere göndermek
- Sistem loglarını tutmak
- Bağlı kullanıcıları GUI üzerinde göstermek

şeklinde sıralanabilir.

5.1. ServerSocket Kullanımı

Sunucunun istemci bağlantılarını kabul edebilmesi için ServerSocket kullanılmaktadır.

Kod:

```
serverSocket = new ServerSocket(port);
```

şeklindedir.

Varsayılan port:

1201

olarak belirlenmiştir.

Sunucu çalışmaya başladığında:

```
Socket socket = serverSocket.accept();
```

koduyla yeni istemci bağlantısı beklenmektedir.

5.2. Çoklu İstemci Desteği

Projenin en önemli özelliklerinden biri aynı anda birden fazla istemcinin desteklenmesidir.

Yeni bir istemci bağlantısı geldiğinde:

```
ClientHandler handler = new ClientHandler(socket);  
clients.add(handler);  
new Thread(handler).start();
```

kodları çalıştırılmaktadır.

Bu yapı sayesinde her istemci için ayrı bir `ClientHandler` nesnesi ve ayrı bir `Thread` oluşturulmaktadır.

Örneğin üç istemci bağlandığında:

```
Client 1 → ClientHandler → Thread 1  
Client 2 → ClientHandler → Thread 2  
Client 3 → ClientHandler → Thread 3
```

şeklinde bağımsız çalışma yapısı oluşmaktadır.

Bu nedenle bir istemcinin ağ üzerinden veri beklemesi diğer istemcilerin bağlantısını doğrudan engellememektedir.

5.3. Client Listesi

Sunucu aktif istemcileri:

```
private final List<ClientHandler> clients =  
    Collections.synchronizedList(new ArrayList<>());
```

yapısı içerisinde tutmaktadır.

`Collections.synchronizedList()` kullanılması ile listenin farklı Thread'ler tarafından erişildiği senaryolarda temel liste işlemlerinin eşzamanlı kullanımına karşı korunması amaçlanmıştır.

Yeni istemci bağlandığında listeye eklenmektedir:

```
clients.add(handler);
```

İstemci ayrıldığında ise listeden çıkarılmaktadır:

```
clients.remove(this);
```

5.4. ClientHandler Sınıfı

`ClientHandler`, sunucuya bağlanan her istemcinin bağlantısını yönetmektedir.

Sınıf:

```
private class ClientHandler implements Runnable
```

şeklinde tanımlanmıştır.

`Runnable` kullanılması sayesinde her istemci bağımsız bir Thread içerisinde çalıştırılabilmektedir.

`ClientHandler` içerisinde:

```
private final Socket socket;  
private DataInputStream dis;  
private DataOutputStream dout;  
private String username;
```

değişkenleri bulunmaktadır.

Bunlar sırasıyla:

- İstemcinin Socket bağlantısını
- Veri okuma akışını
- Veri gönderme akışını
- Kullanıcı adını

tutmaktadır.

5.5. Kullanıcı Adının Alınması

İstemci bağlantı kurduktan sonra ilk olarak kullanıcı adını sunucuya göndermektedir.

Sunucu tarafında bu bilgi:

```
username = dis.readUTF();
```

ile alınmaktadır.

Ardından kullanıcı adı kullanıcı listesine eklenmektedir:

```
SwingUtilities.invokeLater(() ->
    userListModel.addElement(username));
```

Bu sayede sunucu arayüzünde bağlı kullanıcılar görüntülenebilmektedir.

5.6. Broadcast Mekanizması

Sistemde mesajların bağlı istemcilere dağıtılması amacıyla `broadcast()` metodu kullanılmaktadır.

Metot:

```
private void broadcast(String message) {
    synchronized (clients) {
        for (ClientHandler c : clients) {
            c.send(message);
        }
    }
}
```

şeklinde tanımlanmıştır.

Bu mekanizma sayesinde bir istemciden gelen mesaj bağlı istemcilere gönderilmektedir.

Örneğin:

Client A
|

```
| "Merhaba"  
v  
ModernServer  
|  
+----> Client A  
|  
+----> Client B  
|  
+----> Client C
```

şeklinde bir iletişim akışı oluşmaktadır.

5.7. Sistem Mesajları

Sunucu tarafından bağlantı olaylarını bildirmek amacıyla SYS : öneki kullanılmaktadır.

Bir kullanıcı bağlandığında:

```
broadcast("SYS:" + username + " odaya katıldı!");
```

mesajı gönderilmektedir.

Bir kullanıcı ayrıldığında:

```
broadcast("SYS:" + username + " ayrıldı.");
```

mesajı gönderilmektedir.

İstemci tarafı bu mesajları algılayarak sistem mesajı olarak sohbet ekranında göstermektedir.

5.8. Kullanıcı Mesajları

Bir istemciden mesaj geldiğinde sunucu:

```
String msg = dis.readUTF();
```

ile mesajı almaktadır.

Daha sonra mesaj log ekranına yazdırılmaktadır:

```
log(username + ": " + msg);
```

Ardından mesaj:

```
broadcast("MSG:" + username + ":" + msg);
```

kodu ile istemcilere dağıtılmaktadır.

Bu yapı sayesinde mesajın hangi kullanıcı tarafından gönderildiği bilgisi korunmaktadır.

6. SUNUCU GRAFİKSEL ARAYÜZÜ

Sunucu tarafında modern koyu temalı bir kontrol paneli bulunmaktadır.

Arayüz temel olarak üç bölümden oluşmaktadır:

1. Üst kontrol paneli
 2. Sol kullanıcı listesi
 3. Orta sistem log ekranı
-

6.1. Üst Kontrol Paneli

Üst bölümde aşağıdaki bilgiler ve kontroller bulunmaktadır:

- SERVER DASHBOARD başlığı
- Port bilgisi
- Sunucuyu Başlat butonu
- Sunucuyu Kapat butonu
- Çevrimiçi / Çevrimdışı durum bilgisi

Sunucu çalışırken:

- Çevrimiçi (1201)

durumu gösterilmektedir.

Sunucu kapalı olduğunda:

- Çevrimdışı

gösterilmektedir.

6.2. Baęlı Kullanıcılar

Sol tarafta:

BAęLI KULLANICILAR

başıęı altında sunucuya baęlı kullanıcıların listesi bulunmaktadır.

Bu liste:

DefaultListModel<String>

ve:

JList<String>

kullanılarak oluşturulmuştur.

Yeni kullanıcı baęlandığında listeye eklenmekte, baęlantısı kesildięinde listeden kaldırılmaktadır.

6.3. Sistem Logları

Sunucunun gerçekleştirdięi işlemler orta bölümdeki log alanında görüntülenmektedir.

Log kayıtlarında saat bilgisi de tutulmaktadır.

Örnek:

[14:25:10] Sunucu başlatıldı. Port: 1201
[14:25:20] Muhammed (127.0.0.1) baęlandı.
[14:25:30] Muhammed: Merhaba
[14:26:05] Ahmet (127.0.0.1) baęlandı.

Bu yapı sayesinde sunucu yöneticisi sistemde gerçekleşen olayları gerçek zamanlı olarak takip edebilmektedir.

7. RENK VE ARAYÜZ TASARIMI

Projede modern bir koyu tema kullanılmıştır.

Sunucu tarafında kullanılan temel renkler:

COLOR_BG
COLOR_SIDEBAR
COLOR_CARD
COLOR_TEXT
COLOR_ACCENT
COLOR_ONLINE
COLOR_OFFLINE

şeklinde tanımlanmıştır.

İstemci tarafında ise:

COLOR_CHAT_BG
COLOR_TOPBAR
COLOR_ACCENT
COLOR_INPUT

renk değişkenleri kullanılmıştır.

Bu renkler sayesinde uygulamanın farklı bölümleri birbirinden görsel olarak ayrılmış ve modern bir sohbet uygulaması görünümü oluşturulmuştur.

8. KARŞILAŞILAN PROBLEMLER VE ÇÖZÜMLER

8.1. GUI'nin Donması

Ağ işlemlerinin doğrudan Swing'in ana Thread'i üzerinde gerçekleştirilmesi durumunda kullanıcı arayüzü donabilmektedir.

Özellikle:

```
dis.readUTF();
```

işlemi veri gelmesini beklediği için GUI üzerinde çalıştırılması durumunda uygulamanın yanıt vermemesine neden olabilir.

Bu problemi çözmek amacıyla sunucudan mesaj dinleme işlemi ayrı bir Thread içerisinde gerçekleştirilmiştir.

Böylece kullanıcı arayüzü ağ verisini beklemek zorunda kalmamaktadır.

8.2. Tek İstemci Problemi

İlk yaklaşımda sunucu bağlantıyı kabul ettikten sonra istemciyi aynı Thread üzerinde işlediğinde yeni bağlantıların kabul edilmesi engellenebilmektedir.

Bu problem:

```
ClientHandler handler = new ClientHandler(socket);  
new Thread(handler).start();
```

yapısı kullanılarak çözülmüştür.

Böylece her istemci ayrı bir Thread içerisinde çalıştırılmaktadır.

8.3. İstemciler Arası Mesaj Dağıtımı

Bir istemciden alınan mesajın diğer istemcilere gönderilebilmesi için merkezi bir broadcast mekanizmasına ihtiyaç duyulmuştur.

Bu amaçla:

```
broadcast("MSG:" + username + ":" + msg);
```

kullanılmıştır.

Bu yapı sayesinde sunucu, gelen mesajları aktif istemcilere iletebilmektedir.

8.4. Swing Arayüzünün Güvenli Güncellenmesi

Swing bileşenlerinin güncellenmesi için:

```
SwingUtilities.invokeLater(...)
```

kullanılmıştır.

Örneğin kullanıcı listesine yeni kullanıcı eklenirken:

```
SwingUtilities.invokeLater(() ->
    userListModel.addElement(username));
```

şeklinde işlem yapılmaktadır.

Aynı yöntem log ekranının güncellenmesinde de kullanılmaktadır.

9. VERİ İLETİŞİM PROTOKOLÜ

Projede istemci ile sunucu arasındaki mesajların ayrıştırılmasını kolaylaştırmak amacıyla basit bir metin tabanlı protokol kullanılmıştır.

Sistem Mesajı

SYS:[mesaj]

Örnek:

SYS:Muhammed odaya katıldı!

Kullanıcı Mesajı

MSG:[kullanıcı]:[mesaj]

Örnek:

MSG:Muhammed:Merhaba arkadaşlar!

İstemci tarafı mesajın başlangıcındaki SYS : veya MSG : ifadesine göre mesajın türünü belirlemektedir.

10. PROGRAMIN ÇALIŞTIRILMASI

10.1. Sunucunun Çalıştırılması

İlk olarak ModernServer . java çalıştırılır.

Sunucu arayüzünde port numarası kontrol edilir.

Varsayılan değer:

1201

olarak belirlenmiştir.

Daha sonra:

Sunucuyu Başlat

butonuna basılır.

Başarılı şekilde başlatıldığında:

- Çevrimiçi (1201)

ifadesi görüntülenir.

10.2. İstemcinin Çalıştırılması

Sunucu çalıştıktan sonra ModernClient . java çalıştırılır.

İstemci arayüzünde:

IP : 127.0.0.1

Port : 1201

Kullanıcı: Kullanici

bilgileri bulunmaktadır.

Kullanıcı kendi kullanıcı adını yazdıktan sonra:

Bağlan

butonuna basar.

Bağlantı başarılı olduğunda mesaj gönderme alanı aktif hale gelir.

10.3. Çoklu İstemci Testi

Çoklu istemci özelliğini test etmek için ModernClient uygulaması birden fazla kez çalıştırılabilir.

Örnek:

İstemci 1 → Muhammed

İstemci 2 → Ahmet

İstemci 3 → Mehmet

Bu durumda sunucu tarafındaki kullanıcı listesinde:

BAĞLI KULLANICILAR

Muhammed

Ahmet

Mehmet

şeklinde kullanıcılar görüntülenebilir.

Örneğin Muhammed:

Merhaba arkadaşlar!

mesajını gönderdiğinde mesaj sunucuya ulaşır.

Sunucu mesajı:

MSG:Muhammed:Merhaba arkadaşlar!

formatına dönüştürerek bağlı istemcilere gönderir.

11. ÖRNEK ÇALIŞMA SENARYOSU

Sistem başlatıldığında ilk olarak sunucu çalıştırılır.

Sunucu
Port: 1201
Durum: Çevrimiçi

Daha sonra Muhammed isimli kullanıcı bağlanır.

Sunucu logunda:

[14:00:10] Sunucu başlatıldı. Port: 1201
[14:00:20] Muhammed (127.0.0.1) bağlandı.

görüntülenir.

Sunucu ayrıca diğer istemcilere:

SYS:Muhammed odaya katıldı!

mesajını gönderir.

İkinci olarak Ahmet isimli kullanıcı bağlandığında:

[14:01:05] Ahmet (127.0.0.1) bağlandı.

log kaydı oluşturulur.

Sohbet ekranında:

- Ahmet odaya katıldı!

sistem mesajı görüntülenir.

Muhammed mesaj gönderdiğinde:

Merhaba Ahmet!

sunucu bu mesajı alır ve:

MSG:Muhammed:Merhaba Ahmet!

formatında istemcilere gönderir.

İstemciler gelen mesajı kullanıcı adı ve saat bilgisiyle birlikte görüntüler.

12. PROJENİN AVANTAJLARI

Geliştirilen sistemin başlıca özellikleri ve avantajları şunlardır:

- Çoklu istemci desteği
 - TCP tabanlı iletişim
 - Merkezi sunucu mimarisi
 - Her istemci için bağımsız Thread
 - Gerçek zamanlı mesajlaşma
 - Broadcast mekanizması
 - Kullanıcı listesinin gerçek zamanlı güncellenmesi
 - Sunucu log sistemi
 - Modern koyu tema
 - HTML destekli sohbet alanı
 - Kullanıcı adı desteği
 - Saat bilgisi gösterimi
 - Enter tuşuyla mesaj gönderme
 - Bağlantı kurma ve bağlantıyı sonlandırma özelliği
 - Yerel ağ üzerinde çalışabilme
 - Java Swing tabanlı masaüstü arayüz
-

13. PROJENİN SINIRLILIKLARI

Mevcut sürüm temel çoklu istemci sohbet işlevlerini yerine getirmektedir.

Bununla birlikte mevcut yapıda henüz aşağıdaki özellikler bulunmamaktadır:

- Kalıcı mesaj geçmişi
- Kullanıcı hesabı ve kimlik doğrulama
- Şifreli iletişim
- Dosya transferi
- Resim transferi

- Özel mesajlaşma
- Veritabanı desteği
- Gelişmiş kullanıcı yetkilendirme
- TLS/SSL bağlantısı

Bu özelliklerin ilerleyen sürümlerde eklenmesi planlanabilir.

14. GELECEK GELİŞTİRMELER

14.1. Dosya ve Resim Transferi

İstemciler arasında dosya ve resim gönderme özelliği eklenebilir.

Bu özellik sayesinde kullanıcılar yalnızca metin değil, aynı zamanda dosya ve görsel de paylaşabilir.

14.2. Özel Mesajlaşma

Kullanıcıların belirli bir kullanıcıya özel mesaj gönderebilmesi için:

/msg kullanıcı mesaj

benzeri bir komut yapısı eklenebilir.

Örneğin:

/msg Ahmet Merhaba, nasılsın?

şeklinde özel mesaj gönderilebilir.

14.3. SQLite Veritabanı

Mesaj geçmişinin uygulama kapatıldıktan sonra da saklanabilmesi için SQLite veritabanı kullanılabilir.

Veritabanında:

- Kullanıcı adı
- Mesaj
- Tarih
- Saat
- Gönderen bilgisi

gibi veriler saklanabilir.

14.4. TLS/SSL Desteđi

Mevcut TCP iletişimine TLS/SSL desteđi eklenerek istemci ve sunucu arasındaki veri trafiđinin řifrenmesi sađlanabilir.

Bu özellik özellikle ađ üzerinden hassas bilgilerin gönderildiđi durumlarda önem taşımaktadır.

15. SONUÇ

Bu projede Java programlama dili kullanılarak TCP/IP Socket Programming temelli çoklu istemci destekli bir masaüstü sohbet sistemi geliştirilmiştir.

Sistem merkezi istemci-sunucu mimarisine göre tasarlanmış ve sunucu tarafında ServerSocket kullanılarak istemci bağlantılarının kabul edilmesi sađlanmıştır.

Her yeni istemci için ClientHandler nesnesi oluşturulmuş ve bu nesne ayrı bir Thread içerisinde çalıştırılmıştır.

Bu yapı sayesinde sunucu aynı anda birden fazla istemciyi yönetebilmektedir.

İstemci tarafında Socket, DataInputStream ve DataOutputStream kullanılarak sunucu ile veri iletişimi gerçekleştirilmiştir.

Sunucudan gelen mesajların kullanıcı arayüzünü engellememesi amacıyla arka planda çalışan Listener Thread kullanılmıştır.

İstemcilerden gelen mesajların diđer kullanıcılara iletilmesi için broadcast () mekanizması geliştirilmiştir.

Mesajların türlerini birbirinden ayırabilmek için SYS : ve MSG : örneklerinden oluşan basit bir iletişim protokolü kullanılmıştır.

Grafiksel kullanıcı arayüzünde Java Swing bileşenlerinden yararlanılmıştır. Sunucu tarafında bağlı kullanıcılar ve sistem logları görüntülenirken, istemci tarafında HTML destekli JEditorPane kullanılarak modern bir sohbet ekranı oluşturulmuştur.

Ayrıca uygulamada koyu tema, mesaj renkleri, kullanıcı adları ve zaman bilgileri gibi görsel özelliklere yer verilmiştir.

Sonuç olarak proje, Java Socket Programming ve Multi-Threading konularının gerçek bir uygulama üzerinde kullanımını göstermektedir. Geliştirilen mimari, temel bir çoklu kullanıcı sohbet sisteminin oluşturulması için gerekli istemci, sunucu, socket, Thread ve mesaj dağıtım mekanizmalarını bir araya getirmektedir.

Projenin gelecekte dosya transferi, özel mesajlaşma, SQLite veritabanı ve TLS/SSL gibi özelliklerle geliştirilmesi mümkündür.

16. SINIFLARIN GENEL GÖREVLERİ

Sınıf / Bileşen	Görevi
ModernClient	Kullanıcı tarafındaki sohbet uygulamasını yönetir.
ModernServer	Merkezi sunucu uygulamasını yönetir.
ClientHandler	Her bağlı istemcinin bağlantısını bağımsız Thread üzerinde yönetir.
Socket	İstemci-sunucu TCP bağlantısını oluşturur.
ServerSocket	Sunucunun yeni istemci bağlantılarını kabul etmesini sağlar.
DataStream	Soketten gelen verileri okur.
DataOutputStream	Soket üzerinden veri gönderir.
JEditorPane	HTML destekli sohbet mesajlarını görüntüler.
JTextArea	Sunucu loglarını görüntüler.
JList	Bağlı kullanıcıları listeler.
SwingUtilities	Swing GUI güncellemelerinin uygun Thread üzerinde yapılmasını sağlar.

17. PROJE BİLGİLERİ

Proje Adı: Modern Multi-Client Chat Application

Geliştirici: Muhammed Emin Korkunç

E-posta: muhammedeminkorkunc@gmail.com

Programlama Dili: Java

JDK: 21

GUI: Java Swing

Ağ Protokolü: TCP/IP

Varsayılan Port: 1201

Yerel Sunucu Adresi: 127.0.0.1

Paket: javasocketprogramming_java.multiclient.server

18. GELİŞTİRİCİ

Muhammed Emin Korkunç

E-posta: muhammedeminkorkunc@gmail.com

LinkedIn: <https://www.linkedin.com/in/muhammed-emin-korkunc-100ba2215>

GitHub: <https://github.com/muhammedkorkunc>

EK: PROJE DOSYALARI

Projenin temel Java dosyaları:

```
javasocketprogramming_java
├── multiclient
│   └── server
│       └── ModernServer.java
```

└─ ModernClient.java

ModernServer.java sunucu uygulamasını, ModernClient.java ise istemci uygulamasını temsil etmektedir.

Bu iki sınıf birlikte çalışarak çoklu istemci destekli TCP tabanlı sohbet sistemini oluşturmaktadır.